

EfficientNet-B0 학습 파이프라인 사용설명서

노후 건물 균열 3등급(우수/보통/불량) 분류 · MobileNetV2와 불량 F1 성능 비교용 · RunPod 실행 기준

1. 개요

팀원이 만든 MobileNetV2 모델과 성능을 비교하기 위해, 같은 데이터를 EfficientNet-B0로 학습하는 파이프라인입니다. 공정한 비교를 위해 전처리와 데이터 분리(train/validation/test)를 팀원 코드와 완전히 동일하게 맞췄고, 팀원 파일은 한 줄도 수정하지 않았습니다.

팀원과 동일하게 맞추는 것

- 전처리: 224×224 리사이즈 → (train만) 좌우반전·밝기/대비 증감 → float32 스케일 → ImageNet 정규화
- 데이터 분리: 팀원이 만든 metadata_split.csv를 그대로 사용 (재분리하지 않음)
- 클래스 불균형 가중치 [3.79, 2.95, 0.42], Adam 옵티마이저, 2단계 학습(baseline → 파인튜닝)
- 최고 모델 선택 기준: validation 정확도 (선택 조건까지 같아야 모델 차이만 비교됨)

다른 것

- 모델: MobileNetV2 → EfficientNet-B0 (둘 다 ImageNet 사전학습, 224×224 입력)
- 배치 크기: 16 → 128

최종 비교 지표: 프로젝트 목표가 '불량 탐지'이므로 불량을 positive로 본 불량 F1-score가 핵심입니다. 불량 행의 recall(실제 불량을 놓치지 않는 비율)과 precision(불량 판정의 신뢰도)도 함께 봅니다. macro F1과 정확도는 참고용입니다.

2. 파일 구성

모든 코드는 src/efficientnet/ 폴더 안에만 있습니다. 팀원 파일(src/ 바로 아래)은 import조차 하지 않는 완전 독립 구조입니다.

파일	역할
preprocess_efficientnet.py	데이터셋/데이터로더 정의. CSV의 팀원 PC 경로를 data/processed_images/ 경로로 치환해서 읽음. 다른 스크립트가 import해서 사용 (직접 실행하면 데이터 로딩 점검).
train_efficientnet.py	1단계 baseline 학습. 특징 추출부는 동결하고 분류층만 3등급으로 학습.
fine_tune_efficientnet.py	2단계 파인튜닝. baseline을 이어받아 마지막 특징 블록까지 풀고 작은 학습률로 추가 학습. patience 3회 조기 종료.
evaluate_efficientnet.py	Test 데이터 최종 평가. 정확도, 등급별 precision/recall/F1, 혼동행렬 출력 및 파일 저장.

3. 새 RunPod 팟을 켤 때마다 하는 일

팟을 새로 만들면 설치했던 라이브러리와 데이터가 모두 사라집니다. 매번 아래 3단계를 거쳐야 합니다. (컨테이너 ID가 바뀌었다면 새 팟입니다)

① 코드 받기

```
cd /workspace/hn_old-building
git pull
```

처음이라면: `cd /workspace && git clone https://github.com/gpwhdqkr/hn_old-building.git`

② 라이브러리 설치

```
pip install pandas pillow scikit-learn
```

RunPod PyTorch 템플릿이면 torch/torchvision은 보통 이미 있습니다. 확인:

```
python -c "import torch; print(torch.__version__, torch.cuda.is_available())"
```

버전과 True가 나오면 통과. torch가 없다면 pip install torch torchvision 도 실행하세요.

③ 데이터 배치 확인

이미지는 용량 때문에 git에 없습니다. 구글드라이브에서 받은 224×224 이미지를 data/processed_images/ 아래에 풀어야 합니다. (metadata_split.csv는 git에 포함)

```
find data/processed_images -type f | wc -l
```

37285가 나와야 정상입니다. 숫자가 다르면 압축 해제 위치가 잘못된 것입니다 (구조가 data/processed_images/TS_아파트/... 여야 하며, 중간에 폴더가 한 겹 더 있으면 안 됨).

④ (권장) 데이터 로딩 점검 — 1분

```
python src/efficientnet/preprocess_efficientnet.py
```

Train 25947 / Validation 5780 / Test 5558 과 이미지 묶음 형태 torch.Size([128, 3, 224, 224]) 가 나오면 준비 완료입니다.

4. 학습 실행 방법

클라우드는 git pull만 받고 코드를 직접 수정하지 않는다는 원칙에 따라, 설정은 모두 환경변수로 지정합니다. 명령어 앞에 붙이기만 하면 됩니다.

환경변수 3가지

환경변수	기본값	설명
RUN_NAME	default	실험 이름. 저장되는 모델·결과 파일명에 붙어서 실험별로 결과가 따로 쌓임. train / fine_tune / evaluate 에서 같은 값을 써야 이어짐.
NUM_EPOCHS	10	학습 epoch 수 (train과 fine_tune 각각에 적용)
EVAL_TARGET	finetuned	평가할 모델 선택. finetuned 또는 baseline (evaluate에서만 사용)

기본 실행 (epoch 10)

```
nohup python src/efficientnet/train_efficientnet.py > train_log.txt 2>&1 &
tail -f train_log.txt
```

nohup ... & 로 띄우면 터미널이 끊겨도 학습이 계속됩니다. tail -f 에서 빠져나오려면 Ctrl+C (학습은 계속 진행됨).

실험 실행 예시 (epoch 20으로 따로 저장)

```
RUN_NAME=epoch20 NUM_EPOCHS=20 nohup python src/efficientnet/train_efficientnet.py ₩
> train_epoch20.txt 2>&1 &
tail -f train_epoch20.txt
```

train이 끝나면 같은 RUN_NAME으로 파인튜닝:

```
RUN_NAME=epoch20 NUM_EPOCHS=20 nohup python src/efficientnet/fine_tune_efficientnet.py ₩
> finetune_epoch20.txt 2>&1 &
tail -f finetune_epoch20.txt
```

마지막으로 평가:

```
RUN_NAME=epoch20 python src/efficientnet/evaluate_efficientnet.py
```

baseline도 평가하려면:

```
RUN_NAME=epoch20 EVAL_TARGET=baseline python src/efficientnet/evaluate_efficientnet.py
```

실행 직후 반드시 확인할 3가지: ① 실험 이름 (RUN_NAME) 이 의도한 값인지 ② 사용 장치 : cuda 인지 (cpu면 GPU를 못 잡은 것) ③ Epoch 표시가 1/20 처럼 의도한 epoch 수인지. 그리고 로그는 방금 지정한 파일명으로 확인하세요 — 이전 실행의 로그 파일을 열면 옛날 결과가 보여 혼동됩니다.

5. 결과물이 저장되는 위치

결과물	저장 위치	덧어쓰기
baseline 모델	model/best_efficientnet_b0_baseline_<RUN_NAME>.pth	같은 RUN_NAME 재실행 시 덮어씀
파인튜닝 모델	model/best_efficientnet_b0_finetuned_<RUN_NAME>.pth	같은 RUN_NAME 재실행 시 덮어씀
평가 결과 전문	test_results/efficientnet_b0/<RUN_NAME>/evaluate_<대상>_result.txt	같은 RUN_NAME+대상 재평가 시 덮어씀
혼동행렬	test_results/efficientnet_b0/<RUN_NAME>/test_confusion_matrix_<대상>.csv	같은 RUN_NAME+대상 재평가 시 덮어씀

즉 RUN_NAME을 다르게 주면 이전 실험 결과는 안전합니다. 설정을 바꿔 실험할 때는 RUN_NAME을 꼭 새로 지어주세요. F1 등 최종 수치는 evaluate가 txt로 자동 저장하므로 터미널을 닫아도 유실되지 않습니다 (tee 불필요).

팀원 결과물과는 파일명·폴더가 완전히 분리되어 있어 서로 덮어쓸 일이 없습니다. 팀원: model/best_mobilenet_v2_*.pth, test_results/Classical ML/ 등.

6. 결과 해석과 팀원 모델과의 비교

evaluate 실행 시 화면과 txt 파일에 아래 내용이 나옵니다. 팀원의 evaluate_model.py도 같은 형식의 표를 출력하므로 항목끼리 1:1로 비교하면 됩니다 (같은 Test set 5,558장이라 공정 비교).

- Test 정확도 · 맞힌 이미지 수
- 등급별 성능표: 우수/보통/불량 각각의 precision, recall, f1-score
- 혼동행렬: 실제 등급별로 어떤 등급으로 예측했는지 개수 표
- 맨 아래 [비교용 최종 지표]: Test 불량 F1 (핵심), Macro F1-정확도 (참고)

비교 시 볼 것: ① 불량 행의 f1-score(핵심) ② 불량 행의 recall — 실제 불량을 놓치지 않는 비율이라 안전 관점에서 중요 ③ 불량 행의 precision ④ macro avg(참고).

실측 결과 — EfficientNet-B0, epoch 10 (RUN_NAME 미지정)

구분	값
baseline validation 정확도	78.58% (10 epoch, 학습 약 17.5분)
파인튜닝 validation 정확도	82.01% (10 epoch, 학습 약 17.2분)
Test 불량 F1 (핵심)	0.9087 (precision 0.9389 / recall 0.8803)
Test 정확도	81.86% (5,558장 중 4,550장 정답)
Test Macro F1 (참고)	0.6589
등급별 F1	우수 0.6774 / 보통 0.3907 / 불량 0.9087

해석: 목표인 불량은 잘 잡습니다(F1 0.91). 반면 보통 등급이 약점으로 (F1 0.39), 보통 620장 중 447장을 불량으로 오분류합니다. 데이터가 불량 4,437장 대 보통 620장으로 크게 치우친 영향으로 보입니다. 다만 실제 불량을 '우수'로 오판한 경우는 84장뿐이라, 심각한 놓침은 상대적으로 적습니다.

7. 문제 해결 (실제로 겪은 것들)

■ ModuleNotFoundError: No module named 'sklearn' (또는 pandas)

새 팟이라 라이브러리가 없는 경우입니다. pip install pandas pillow scikit-learn 실행. 설치 시 패키지명은 sklearn이 아니라 scikit-learn입니다.

■ 학습이 비정상적으로 느리고 nvidia-smi의 GPU-Util이 5% 수준

CPU 데이터 로딩이 병목입니다. 현재 코드는 num_workers를 CPU 코어 수에 맞춰 자동 계산(코어 절반, 최소 4 최대 32)하므로 최신 코드를 git pull 받으면 해결됩니다. 확인은 nvidia-smi로 GPU-Util이 70% 이상 나오는지 보면 됩니다.

■ 로그에 이전 실험 결과가 보임 / 학습이 안 되는 것처럼 보임

이전 실행이 남긴 다른 로그 파일을 보고 있는 경우입니다. 이번에 지정한 파일명(예: train_epoch20.txt)으로 tail -f 하세요.

■ metadata_split.csv 또는 이미지 폴더를 찾을 수 없습니다

프로젝트 폴더(/workspace/hn_old-building)에서 실행 중인지, data/processed_images 이미지가 37,285장인지 확인하세요.

■ baseline 모델을 찾을 수 없습니다

train을 건너뛰었거나 RUN_NAME이 달라서 못 찾는 경우입니다. train 때와 같은 RUN_NAME을 지정했는지 확인하세요. (에러 메시지에 찾던 경로가 표시됨)

■ CUDA out of memory

preprocess_efficientnet.py의 batch_size(128)를 줄여야 합니다. 배치 크기는 학습 속도에만 영향을 주며 비교의 공정성에는 문제없습니다.

■ 사용 장치 : cpu 로 나눔

GPU를 못 잡은 상태입니다. 그대로 두면 학습이 몇십 배 느려지니 중단하고, torch가 CUDA 빌드인지 확인하세요 (pip install torch torchvision --index-url <https://download.pytorch.org/whl/cu126>).

8. 결과물 깃 관리 주의사항

대상	권장	이유
test_results/ (txt, CSV)	푸시 권장	몇 KB 수준이고 비교 근거 자료라 팀 공유 가치가 큼
최종 채택 모델 .pth	선별 푸시 가능	EfficientNet-B0 기준 개당 약 16~20MB. GitHub 한도(100MB)에는 여유
실험 중간 모델 .pth	푸시 비권장	깃은 히스토리를 영구 보관하므로 실험마다 커밋하면 리포가 계속 불어남. 수치는 test_results의 txt에 남으니 필요 시 재학습
data/processed_images /	푸시 금지	이미 .gitignore에 등록됨. 구글드라이브로 공유

푸시 전에는 항상 git status로 무엇이 올라가는지 확인하세요. __pycache__ 같은 캐시 폴더는 커밋하지 않습니다.

9. 전체 흐름 요약 (복사해서 쓰는 용도)

```
# 0) 준비 (새 팟마다 1회)
cd /workspace/hn_old-building
git pull
pip install pandas pillow scikit-learn
find data/processed_images -type f | wc -l # 37285 확인

# 1) 데이터 로딩 점검
python src/efficientnet/preprocess_efficientnet.py

# 2) baseline 학습
RUN_NAME=exp1 NUM_EPOCHS=10 nohup python src/efficientnet/train_efficientnet.py ₩
> train_exp1.txt 2>&1 &
tail -f train_exp1.txt

# 3) 파인튜닝 (2번 완료 후)
RUN_NAME=exp1 NUM_EPOCHS=10 nohup python src/efficientnet/fine_tune_efficientnet.py ₩
> finetune_exp1.txt 2>&1 &
tail -f finetune_exp1.txt

# 4) 평가 (결과는 자동 저장됨)
RUN_NAME=exp1 python src/efficientnet/evaluate_efficientnet.py
RUN_NAME=exp1 EVAL_TARGET=baseline python src/efficientnet/evaluate_efficientnet.py

# 5) 결과 확인
cat test_results/efficientnet_b0/exp1/evaluate_finetuned_result.txt
```

대상 코드: src/efficientnet/ (4개 파일) · 실행 환경: RunPod Ubuntu, GPU 20GB VRAM 기준 · 팀원 코드(src/ 바로 아래)는 수정하지 않음